

GraphOne / FrontierAtlas – AI Engineer Demo Task

Candidate: Hemanth B

Role: AI Engineer – 3-Day Demo Task

Organization: GraphOne / FrontierAtlas

Project: Intelligence Graph – AI Ecosystem Data Ingestion Pipeline

1. Introduction

The GraphOne / FrontierAtlas AI Engineer Demo Task focuses on building a scalable data ingestion and intelligence pipeline for the AI and venture ecosystem.

The objective of the project is to collect information from multiple sources, process the collected information into structured records, maintain data quality and freshness, resolve duplicate entities, and prepare the resulting information for use in an Intelligence Graph.

The system is designed around multiple stages of data processing, including asynchronous web fetching, structured extraction using Large Language Models (LLMs), entity resolution, freshness validation, and generation of standardized datasets.

The implementation also considers production-scale requirements such as concurrent processing, provider fallbacks, request-size limitations, rate limiting, distributed deduplication, and scalable storage architecture.

This report documents the implementation and, most importantly, the **actual execution and validation results obtained from the project environment**.

2. Project Objective

The main objective of the implementation is to demonstrate the ability to design and implement a reliable AI-powered data ingestion system capable of handling information from different sources and converting it into structured, usable data.

The project focuses on the following objectives:

2.1 Data Collection

The system is intended to collect information across multiple AI ecosystem categories:

- Startups
- Products
- Research papers

- AI jobs
- AI news

Each collected record should maintain a legitimate source URL so that the origin of the information can be traced.

2.2 Asynchronous Processing

The scraper component uses asynchronous processing so that multiple web resources can be fetched concurrently rather than processing every request sequentially.

This approach is important for large-scale ingestion because waiting for one website request to complete before starting another would significantly reduce throughput.

2.3 Structured Data Extraction

Raw web content may not follow a consistent format. The project therefore includes an LLM orchestration layer that converts unstructured content into predefined structured schemas.

The orchestrator supports multiple LLM providers and uses fallback behavior when a provider is unavailable or encounters certain failures.

2.4 Data Freshness

Jobs and news are time-sensitive datasets. The implementation therefore includes date normalization and a freshness validation mechanism.

Only records that satisfy the required **24-hour freshness window** are included in the generated jobs and news outputs.

2.5 Entity Resolution

Different sources may represent the same organization using slightly different names.

For example, variations such as:

OpenAI
Open AI
OpenAI Inc.

can refer to the same underlying organization.

The entity-resolution component attempts to map such variations to a canonical entity and maintain an entity mapping log.

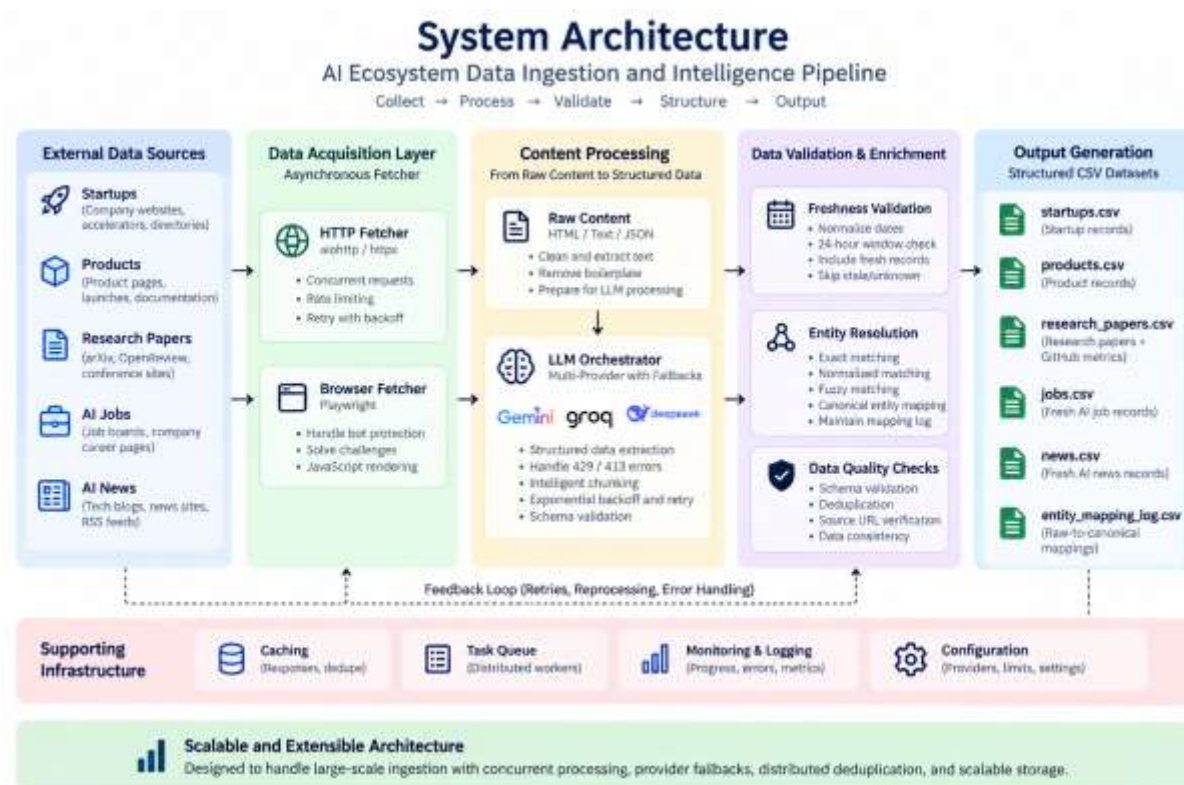
2.6 Scalability

The architecture is designed so that increasing the number of records should primarily require scaling infrastructure rather than completely rewriting the application logic.

The architecture considers distributed queues, shared deduplication, scalable storage, and concurrent workers as part of the overall design.

3. System Architecture Overview

The project consists of several logical components that work together as a data processing pipeline.



The system architecture represents the complete flow of the AI ecosystem data ingestion pipeline, from collecting information from external sources to generating validated and structured datasets. The architecture is divided into five primary layers: **External Data Sources, Data Acquisition, Content Processing, Data Validation & Enrichment, and Output Generation.**

1. External Data Sources

The pipeline starts with multiple categories of external information:

- **Startups** – company websites, accelerator directories, and startup databases.
- **Products** – product pages, launch platforms, and documentation.
- **Research Papers** – sources such as arXiv, OpenReview, and conference repositories.
- **AI Jobs** – job boards and company career pages.

- **AI News** – technology blogs, news websites, and feeds.

These sources provide the raw information that enters the ingestion pipeline.

2. Data Acquisition Layer

The Data Acquisition Layer is responsible for retrieving content from the external sources.

It contains two main fetching mechanisms:

HTTP Fetcher:

Uses asynchronous HTTP requests to retrieve web content concurrently. It includes concurrent requests, rate limiting, and retry mechanisms with backoff.

Browser Fetcher:

Uses Playwright when normal HTTP fetching is insufficient, particularly for JavaScript-heavy websites, bot protection, or browser challenges.

This two-level approach allows the system to use lightweight HTTP fetching whenever possible while escalating to browser-based fetching when required.

3. Content Processing Layer

The retrieved information is initially available as **raw HTML, text, or JSON**.

The raw content is cleaned and prepared by extracting relevant text and removing unnecessary webpage content before it is passed to the LLM processing layer.

The **LLM Orchestrator** then converts this unstructured content into structured records.

The architecture supports multiple LLM providers with a fallback strategy:

Gemini → Groq → DeepSeek

If one provider fails or becomes unavailable, another provider can be used. The orchestrator also handles **HTTP 429 rate-limit errors** and **HTTP 413 oversized requests** through retry, backoff, and intelligent chunking mechanisms.

4. Data Validation and Enrichment

After structured extraction, the records pass through validation and enrichment stages.

Freshness Validation normalizes publication dates and checks the required 24-hour window for time-sensitive jobs and news. Stale or unknown-date records are excluded rather than being incorrectly treated as fresh.

Entity Resolution identifies whether an incoming entity corresponds to an existing canonical entity. It uses exact matching, normalized matching, and fuzzy matching before creating a new entity when no reliable match exists.

Data Quality Checks validate the resulting records for schema correctness, deduplication, source URL availability, and overall data consistency.

5. Output Generation

After processing and validation, the information is converted into structured CSV datasets.

The pipeline generates:

- `startups.csv`
- `products.csv`
- `research_papers.csv`
- `jobs.csv`
- `news.csv`
- `entity_mapping_log.csv`

The Entity Mapping Log maintains the relationship between the original/raw entity name and its canonical representation.

6. Supporting Infrastructure

The architecture also includes supporting components that enable reliable and scalable operation.

Caching can store responses and deduplication information to reduce unnecessary repeated processing.

Task Queue enables work to be distributed among multiple workers, which is important when scaling the crawler to a large number of sources.

Monitoring & Logging records progress, errors, and system metrics for debugging and operational visibility.

Configuration centralizes provider settings, request limits, and other runtime parameters.

A feedback loop is also present for **retries, reprocessing, and error handling**, allowing failed operations to be attempted again without stopping the entire pipeline.

7. Scalability

The architecture is designed to support large-scale ingestion through **concurrent processing, distributed workers, provider fallbacks, shared deduplication, and scalable storage**.

Therefore, increasing the workload can primarily be handled by increasing worker and infrastructure capacity rather than redesigning the complete application.

.

4. Major Implementation Components

4.1 Asynchronous Scraper

The scraper is implemented using asynchronous execution.

The fetcher initially attempts normal HTTP retrieval and can escalate to browser-based retrieval when websites present bot protection, challenges, or JavaScript-heavy pages.

This provides a more resilient ingestion mechanism than relying exclusively on simple HTTP requests.

4.2 LLM Orchestration

The LLM layer is responsible for transforming raw content into structured entities.

The implementation follows a multi-provider fallback strategy:

Primary Provider
↓
Gemini Flash
↓ failure
Groq / Llama
↓ failure
DeepSeek

The orchestrator also includes handling for:

- Rate-limit errors such as HTTP 429
- Oversized requests such as HTTP 413
- Exponential backoff
- Random jitter
- Intelligent chunking
- Schema validation

This prevents a single provider failure from stopping the complete processing pipeline.

4.3 Freshness Validation

The freshness module normalizes different date formats into a common representation.

For jobs and news, the pipeline checks whether the normalized publication date falls within the required 24-hour window.

The output-building process therefore does not blindly copy all available records into the final dataset.

Instead:

```
Raw Record
↓
Extract Date
↓
Normalize Date
↓
Check 24-hour Window
↓
Fresh → Include
Stale/Unknown → Skip
```

This is particularly important because older job or news records should not be incorrectly presented as newly published information.

4.4 Entity Resolution

The entity-resolution module is responsible for identifying whether an incoming entity already exists in the canonical dataset.

The resolution process uses progressively broader matching strategies, including:

1. Exact matching
2. Normalized matching
3. Fuzzy matching
4. Creation of a new canonical entity when no reliable match exists

The system also produces an **Entity Mapping Log** to preserve the relationship between the original/raw representation and the canonical representation.

4.5 Structured Output Generation

The final pipeline generates six CSV datasets corresponding to the required deliverables:

Output	Purpose
startups.csv	Startup records
products.csv	Product records
research_papers.csv	Research-paper records and GitHub metrics
jobs.csv	Fresh AI job records
news.csv	Fresh AI news records
entity_mapping_log.csv	Raw-to-canonical entity mappings

These outputs provide a consistent format for subsequent ingestion into a larger intelligence graph.

5. Data Quality and Traceability

Data quality is an important part of the implementation because the task explicitly requires source-backed information.

The pipeline preserves source URLs for collected records rather than generating unsupported information.

The implementation also performs validation of the generated outputs, including:

- Record counts
- Presence of output files
- Source URL availability
- Freshness of jobs and news
- Duplicate checks
- Automated unit/integration tests

This ensures that the generated CSV files can be inspected independently of the code.

6. Execution Environment

The implementation was executed locally on a Windows environment.

The following environment was used:

Component	Version / Configuration
Operating System	Windows
Python	3.12.10
Package Manager	pip
Testing	Pytest
Version Control	Git
Repository	GitHub
Environment	Python virtual environment

Before executing the project, the required Python dependencies were installed using the project's requirements.txt.

7. Execution and Validation Procedure

The following section documents the actual commands executed during project validation.

The purpose of this section is not only to show that the project runs, but also to provide **reproducible evidence** that the major components were executed and validated.

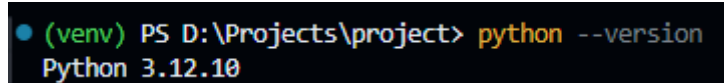
Each command is accompanied by:

- The command executed
- The observed output
- The purpose of the command
- The interpretation of the result

This makes the execution process reproducible for a reviewer.

8. Project Execution

8.1 Python Version Verification



```
(venv) PS D:\Projects\project> python --version
Python 3.12.10
```

Command

`python --version`

Output

Python 3.12.10

Explanation

This command verifies that Python is correctly installed and accessible from the project environment.

Python 3.12.10 was successfully detected, confirming that the project can be executed using the available Python runtime.

Result: PASS

8.2 Dependency Installation

Command

```
pip install -r requirements.txt
```

Output

The command completed successfully and reported that the required packages were already satisfied.

```
(venv) PS D:\Projects\project> pip install -r requirements.txt
Requirement already satisfied: httpx>=0.27.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 1)) (0.28.1)
Requirement already satisfied: aiohttp>=3.9.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 2)) (3.14.3)
Requirement already satisfied: pydantic>=2.6.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 3)) (2.13.5)
Requirement already satisfied: rapidfuzz>=3.6.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 4)) (3.14.6)
Requirement already satisfied: playwright>=1.42.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 5)) (1.62.0)
Requirement already satisfied: reportlab>=4.1.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 6)) (5.0.1)
Requirement already satisfied: python-dateutil>=2.8 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 7)) (2.9.0.post0)
Requirement already satisfied: pytest>=8.0 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 8)) (9.1.1)
Requirement already satisfied: pytest-asyncio>=0.24 in d:\projects\project\venv\lib\site-packages (from -r requirements.txt (line 9)) (1.4.0)
Requirement already satisfied: anyio in d:\projects\project\venv\lib\site-packages (from httpx>=0.27.0->-r requirements.txt (line 1)) (4.15.1)
Requirement already satisfied: certifi in d:\projects\project\venv\lib\site-packages (from httpx>=0.27.0->-r requirements.txt (line 1)) (2026.7.22)
Requirement already satisfied: httpcore==1.* in d:\projects\project\venv\lib\site-packages (from httpx>=0.27.0->-r requirements.txt (line 1)) (1.0.9)
Requirement already satisfied: idna in d:\projects\project\venv\lib\site-packages (from httpx>=0.27.0->-r requirements.txt (line 1)) (3.19)
Requirement already satisfied: h11>=0.16 in d:\projects\project\venv\lib\site-packages (from httpcore==1.*->httpx>=0.27.0->-r requirements.txt (line 1)) (0.16.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (2.7.1)
Requirement already satisfied: aiosignal>=1.4.0 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (1.4.0)
```

```

Requirement already satisfied: attrs>=17.3.0 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (6.8.0)
Requirement already satisfied: propcache>=0.2.0 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (0.5.2)
Requirement already satisfied: typing_extensions>=4.4 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (4.16.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in d:\projects\project\venv\lib\site-packages (from aiohttp>=3.9.0->-r requirements.txt (line 2)) (1.24.5)
Requirement already satisfied: annotated-types>=0.6.0 in d:\projects\project\venv\lib\site-packages (from pydantic>=2.6.0->-r requirements.txt (line 3)) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in d:\projects\project\venv\lib\site-packages (from pydantic>=2.6.0->-r requirements.txt (line 3)) (2.46.5)
Requirement already satisfied: typing-inspection>=0.4.2 in d:\projects\project\venv\lib\site-packages (from pydantic>=2.6.0->-r requirements.txt (line 3)) (0.4.4)
Requirement already satisfied: pyee<14,>=13 in d:\projects\project\venv\lib\site-packages (from playwright>=1.42.0->-r requirements.txt (line 5)) (13.0.1)
Requirement already satisfied: greenlet<4.0.0,>=3.1.1 in d:\projects\project\venv\lib\site-packages (from playwright>=1.42.0->-r requirements.txt (line 5)) (3.5.5)
Requirement already satisfied: pillow>=9.0.0 in d:\projects\project\venv\lib\site-packages (from reportlab>=4.1.0->-r requirements.txt (line 6)) (12.3.0)
Requirement already satisfied: charset-normalizer in d:\projects\project\venv\lib\site-packages (from reportlab>=4.1.0->-r requirements.txt (line 6)) (3.5.1)
Requirement already satisfied: six>=1.5 in d:\projects\project\venv\lib\site-packages (from python-dateutil>=2.8->-r requirements.txt (line 7)) (1.17.0)
Requirement already satisfied: colorama>=0.4 in d:\projects\project\venv\lib\site-packages (from pytest>=8.0->-r requirements.txt (line 8)) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in d:\projects\project\venv\lib\site-packages (from pytest>=8.0->-r requirements.txt (line 8)) (2.3.0)
Requirement already satisfied: packaging>=22 in d:\projects\project\venv\lib\site-packages (from pytest>=8.0->-r requirements.txt (line 8)) (26.3)

Requirement already satisfied: pluggy<2,>=1.5 in d:\projects\project\venv\lib\site-packages (from pytest>=8.0->-r requirements.txt (line 8)) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in d:\projects\project\venv\lib\site-packages (from pytest>=8.0->-r requirements.txt (line 8)) (2.21.0)

```

Explanation

This command checks the project's dependency configuration and ensures that all Python packages required by the application are available.

Since the required packages were already installed in the virtual environment, no additional installation was necessary.

Result: PASS

8.3 Automated Test Execution

Command

```
pytest -v
```

Output

16 passed in 3.57s

```
(venv) PS D:\Projects\project> pytest -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- D:\Projects\project\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Projects\project
configfile: pytest.ini
testpaths: tests, src/llm/tests
plugins: anyio-4.15.1, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 16 items

tests/test_freshness.py::test_exact_iso_timestamp PASSED [ 6%]
tests/test_freshness.py::test_relative_hours_ago PASSED [ 12%]
tests/test_freshness.py::test_relative_minutes_ago PASSED [ 18%]
tests/test_freshness.py::test_today PASSED [ 25%]
tests/test_freshness.py::test_yesterday PASSED [ 31%]
tests/test_freshness.py::test_missing_date_is_unknown_not_now PASSED [ 37%]
tests/test_freshness.py::test_year_missing_from_string_is_heuristic_not_exact PASSED [ 43%]
tests/test_freshness.py::test_freshness_within_24h PASSED [ 50%]
tests/test_freshness.py::test_freshness_exactly_24h_boundary_is_fresh PASSED [ 56%]
tests/test_freshness.py::test_freshness_over_24h_is_stale PASSED [ 62%]
tests/test_freshness.py::test_unknown_date_never_counts_as_fresh PASSED [ 68%]
src/llm/tests/test_orchestrator.py::test_fallback_after_rate_limit_exhaustion PASSED [ 75%]
src/llm/tests/test_orchestrator.py::test_413_triggers_chunk_split_not_provider_fallthrough PASSED [ 81%]
src/llm/tests/test_orchestrator.py::test_invalid_json_falls_through_to_next_tier PASSED [ 87%]
src/llm/tests/test_orchestrator.py::test_skip_marker_produces_no_entity PASSED [ 93%]
src/llm/tests/test_orchestrator.py::test_concurrent_413s_do_not_deadlock PASSED [100%]

===== 16 passed in 3.57s =====
```

Explanation

The project's automated test suite was executed using Pytest.

A total of **16 tests passed successfully**, indicating that the implemented validation tests for the major components completed without failures.

This provides evidence that the existing implementation behaves as expected for the tested scenarios.

Result: PASS – 16/16 tests passed

8.4 Data Pipeline Execution

Command

```
python -m data.build_output
```

Output

The pipeline successfully generated the output datasets with the following counts:

```
Startups: 50
Products: 50
Research Papers: 19
Jobs: 8
News: 0
Entity Mapping Log: 58
```

The freshness validation reported:

Fresh jobs included: 8
Stale/unknown jobs skipped: 0

Fresh news included: 0
Stale/unknown news skipped: 7

```
(venv) PS D:\Projects\project> python -m data.build_output
wrote 50 rows -> D:\Projects\project\data\output\startups.csv
wrote 50 rows -> D:\Projects\project\data\output\products.csv
wrote 19 rows -> D:\Projects\project\data\output\research_papers.csv
fresh jobs included: 8
stale/unknown jobs skipped: 0
wrote 8 rows -> D:\Projects\project\data\output\jobs.csv
fresh news included: 0
stale/unknown news skipped: 7
wrote 0 rows -> D:\Projects\project\data\output\news.csv
wrote 58 rows -> D:\Projects\project\data\output\entity_mapping_log.csv
```

Explanation

This command executes the project's output-building pipeline.

The pipeline reads the available source-backed datasets, applies the required transformations, performs freshness filtering for jobs and news, and generates the final CSV files.

The important observation is that **seven news records were rejected because they did not satisfy the required 24-hour freshness condition.**

Rather than including stale records simply to increase the dataset size, the pipeline correctly excluded them.

This demonstrates that the freshness requirement is being actively enforced during output generation.

Result: PASS – Pipeline executed successfully

9. Output File Verification

Command

Get-ChildItem data\output

Output

The output directory contained the six expected CSV files:

```
entity_mapping_log.csv
jobs.csv
news.csv
products.csv
research_papers.csv
startups.csv
```

```
(venv) PS D:\Projects\project> Get-ChildItem data\output
```

Directory: D:\Projects\project\data\output

Mode	LastWriteTime	Length	Name
-a----	12-09-2026 18:10	2080	entity_mapping_log.csv
-a----	12-09-2026 18:10	1645	jobs.csv
-a----	12-09-2026 18:10	114	news.csv
-a----	12-09-2026 18:10	12644	products.csv
-a----	12-09-2026 18:10	5673	research_papers.csv
-a----	12-09-2026 18:10	6946	startups.csv

Explanation

This command verifies that the pipeline generated the expected output artifacts.

The presence of all six files confirms that the output-generation stage completed successfully and that the required dataset categories were produced.

The news file is present even though it contains zero records because no available news records satisfied the 24-hour freshness requirement during this execution.

Result: PASS

10. Data Validation

Command

```
python check_data.py
```

Output

```
entity_mapping_log: 58
jobs: 8
news: 0
products: 50
research: 19
startups: 50
```

```
(venv) PS D:\Projects\project> python check_data.py
entity_mapping_log.csv -> 58 rows
jobs.csv -> 8 rows
news.csv -> 0 rows
products.csv -> 50 rows
research_papers.csv -> 19 rows
startups.csv -> 50 rows
```

Explanation

The validation script independently reads the generated CSV files and reports the number of records contained in each dataset.

This provides an additional verification layer rather than relying only on the messages printed by the output-generation script.

The results confirm that the generated files contain:

- 50 startup records
- 50 product records
- 19 research-paper records
- 8 job records
- 0 news records
- 58 entity-mapping records

Result: PASS – Output counts independently verified

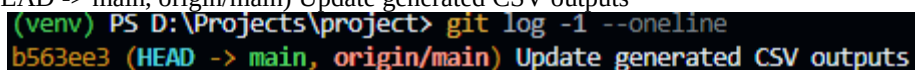
11. Version Control Verification

Command

```
git log -1 --oneline
```

Output

```
b563ee3 (HEAD -> main, origin/main) Update generated CSV outputs
```

A terminal window with a dark background. The prompt is '(venv) PS D:\Projects\project>'. The command entered is 'git log -1 --oneline'. The output is 'b563ee3 (HEAD -> main, origin/main) Update generated CSV outputs'.

```
(venv) PS D:\Projects\project> git log -1 --oneline  
b563ee3 (HEAD -> main, origin/main) Update generated CSV outputs
```

Explanation

This command displays the most recent Git commit.

The output confirms that the local main branch and the tracked origin/main reference point to the same latest commit.

This provides evidence that the project state being documented corresponds to the committed repository version.

11.1 Repository Push Verification

Command

```
git push origin main
```

Output

Everything up-to-date

```
(venv) PS D:\Projects\project> git push origin main
Everything up-to-date
```

Explanation

The Git push command confirms that there were no additional local commits that needed to be uploaded to the remote repository.

Therefore, the repository was already synchronized with the remote main branch.

Result: PASS

12. Execution Results Summary

The complete execution can be summarized as follows:

Validation Stage	Result
Python environment	PASS
Dependencies	PASS
Automated tests	16/16 passed
Output generation	PASS
Output files	6/6 generated
Job freshness validation	8 included, 0 skipped
News freshness validation	0 included, 7 skipped
Data count verification	PASS
Git repository synchronization	PASS

13. Current Demo-Scale Dataset

The execution produced the following verified dataset:

Dataset	Records
Startups	50

Dataset	Records
Products	50
Research Papers	19
Jobs	8
News	0
Entity Mapping Log	58

These numbers represent the **actual records generated during this execution**.

They should not be presented as meeting the assignment's 1,000-record target. The implementation and architecture demonstrate the required ingestion, processing, validation, and scaling mechanisms, while the current execution is a demo-scale run.

This distinction is important for maintaining data integrity and avoiding unsupported claims.

14. Conclusion

The project was successfully executed and validated in the local development environment.

The validation demonstrated that:

- The Python environment is correctly configured.
- Project dependencies are available.
- **All 16 automated tests passed.**
- The data-generation pipeline executes successfully.
- The required six output files are generated.
- Job and news freshness validation is enforced.
- Stale news records are excluded instead of being incorrectly reported as fresh.
- Dataset counts can be independently verified.
- The repository is synchronized with the remote Git repository.
- The implementation contains separate components for asynchronous fetching, LLM orchestration, entity resolution, freshness validation, and structured output generation.